॥ श्रीगणेशाय नमः ॥

Arrays

Array

An array is a data structure consisting of a collection of elements, of same memory size, each identified by at least one array index or key.
In most programming languages, array indexing starts from 1. For discussing algorithms, the starting index is often considered other than 0.

In C programming:

```
int a[] = { 10, 20, 30, 40 };
// or
int a[10];
```

Addressing

Consider an array having each element of size 2B with the starting address (base) as 1000 and the last address 1019.



$\#\text{ of addresses} = \frac{(1019+1) - 1000}{2} = 10$ (2-1 i.e. size-1)

To find the address of the 7th element:

location(7) = base address + (7-1) * 2 (size of each element; # of elements to cross to reach the 7th element)



size of each element = 5

loc(A[278]) = ?

of elements = 300 - 95 + 1 = 206

Base address = 1000

loc(A[278])

95, 96, ..., 277, 278

(277 - 95 + 1) elements to cross = (278 - 95) (reqd. index - base index)

loc(A[278]) = BA + (278-95)·5
= 1000 + (278-95)5
= 1915

Q. A[-300, ..., 500] BA = 0. size of each element = 10. loc(A[-4]) = ?

→ (-5) - (-300) + 1 = (-4) - (-300) = 296

296·10 + BA = 2960

Formula for addressing

A[lb ... ub]

location of A[i] = BA + (i - lb)·size

i = reqd. index, lb = lower bound, ub = upper bound, size = size of each element

Why do programming languages use the starting index as 0?

It simplifies the formulation for the address as lb = 0, reducing the time to calculate the address.

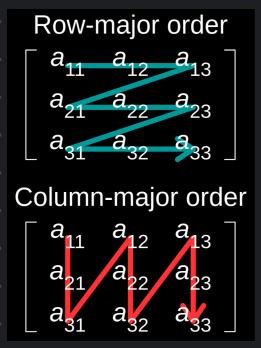
BA + (i - lb)·size = BA + i·size

E.g. loc(A[7]) = B + 7·size

2-D arrays

```
int a[][3] = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
```

↳ atleast the column-size must be specified when defining like this (C).



2-D arrays are stored like 1-D arrays, so the way they are read could be row-major or column major.

Row major

	1	2	3	4
1	10	20	30	40
2	50	60	70	80
3	90	100	110	120
4	130	140	150	160

1000	1002	1004	1006	1008	1010	1012	1014	1016	1018	1020	1022	1024	1026	1028	1030
10	20	30	40	50	60	70	80	90	100	110	120	130	140	150	160

$A[1 \cdots 4, 1 \cdots 4]$

Address of [4, 3]:

10 20 30 40
50 60 70 80
90 100 110 120
130 140 150 160

cross 3 rows = 3 * 4 elements (4 - 1)

then 2 elements (3 - 1)

of element in each row = # of cols in array

Address (A[4][3]) = $1000 + [(4-1) \cdot 4 + (3-1)] \cdot \text{size}$

(# of elements in each row)

Q. A[27, ..., 71][43, ..., 120]. Size of each element = 10. BA = 1000. Row-major addressing is used. Address of A[50][110].

→ # of rows = 71 - 27 + 1 = 45

of columns = 120 - 43 + 1 = 78

$BA + [(50-27) \cdot 78 + (110-43)] \cdot 10 = 19610.$

A: 27 ... 71 × 43 ... 120

of elements in 1 row = # of columns = 78

crossing 50 - 27 rows

[50][110]

crossing 110 - 43 elements

Q. A[-93, ..., 100][-25, ..., 25]. B = 0, size = 1, row-major. Address of A[95][20] = ?

→ # of cols = 51

Address = $0 + [(95 - (-93)) \cdot 51 + (20 - (-25))] \cdot 1 = 9633.$

Formula for row major addressing

$A[l_1 \cdots u_1][l_2 \cdots u_2]$.

of columns = $(u_2 - l_2 + 1)$

Address of $A[i][j] = BA + [(i - l_1) \cdot c + (j - l_2)] \cdot \text{size}$

Column major

	1	2	3	4	5
1	a_{11}	a_{12}	a_{13}	a_{14}	a_{15}
2	a_{21}	a_{22}	a_{23}	a_{24}	a_{25}
3	a_{31}	a_{32}	a_{33}	a_{34}	a_{35}
4	a_{41}	a_{42}	a_{43}	a_{44}	a_{45}

1000	1002	1004	1006	1008	1010	1012	1014	1016	1018	1020	1022	1024	1026	1028	1030	1032	1034	1036	1038
a_{11}	a_{21}	a_{31}	a_{41}	a_{12}	a_{22}	a_{32}	a_{42}	a_{13}	a_{23}	a_{33}	a_{43}	a_{14}	a_{24}	a_{34}	a_{44}	a_{15}	a_{25}	a_{35}	a_{45}

4 columns × 3 elements in each (# of rows) = (5 - 1)

2 elements

A[3][5]

Location (A[3][5]) = $1000 + [(5-1) \cdot 4 + (3-1)] \cdot 2 = 1036$

(5 - 1) 4 + (3 - 1)

of rows = $u_1 - l_1 + 1$

Address of $A[i][j] = BA + [(j - l_2)(r) + (i - l_1)] \cdot \text{size}$

Formula for column major addressing

Q. A[25, ..., 97][-16, ..., 200] BA = 0, size = 5, CMO. Loc (A[90][180]) = ?

↳ # of rows = 97 - 25 + 1 = 73

→ A[90][180]: $0 + [(180 - (-16)) \cdot 73 + (90 - 25)] \cdot 5 = 71865$

Q. A[-42, ..., 42][-84, ..., 84]. BA = 1000. size = 10. CMO. Loc (A[-3][-4]).

↳ # of rows = 85 ↳ # of cols = 165.

→ A[-3][-4]: $1000 + [(-4 - (-84)) \cdot 85 + (-3 - (-42))] \cdot 10 = 69390$

For a multidimensional array, the element with indices i,j would have address $B + c \cdot i + d \cdot j$, where the coefficients c and d are the *row* and *column address increments*, respectively.

More generally, in a k-dimensional array, the address of an element with indices $i_1, i_2, \ldots, i_k$ is

$B + c_1 \cdot i_1 + c_2 \cdot i_2 + \ldots + c_k \cdot i_k$.

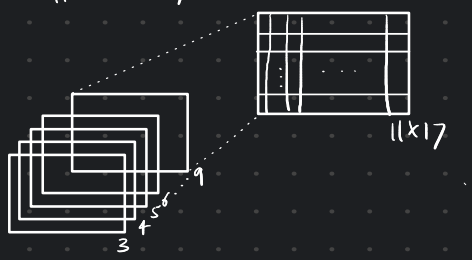
For 3-D arrays

specifying these is necessary in C.

```
int a[][][] = { 1, 2, 3, ... };
```

$A[3 \ldots 9][5 \ldots 15][8 \ldots 24]$. $B = 1000$, size $= 2$.

c_7 c_{11} c_{17}



loca^n of $A[7][12][20]$:

we need to cross 3, 4, 5, 6 2-D arrays to get to 7.

Each has 11×17 elements

Then the $(12-5)$ rows, then $(20-8)$ elements

$[(7-3) \cdot 11 \cdot 17 + (12-5) \cdot 17 + (20-8)]$ total elements to cross

loca^n of $A[7][12][20] = 1000 + [(7-3) \cdot 11 \cdot 17 + (12-5) \cdot 17 + (20-8)] \cdot 2$

$= 2758$

$A[3 \ldots 9][5 \ldots 15][8 \ldots 24]$. BA $= 1000$, size $= 2$. CMO. Find loc($A[7][12][20]$).

c_7 c_{11} c_{17}

$7-3 = 4$ $\quad 12-5 = 7$ $\quad 20-8 = 12$

address $= 1000 + [4 \cdot 11 \cdot 7 + 12 \cdot 11 + 7] \cdot 2$

Q. $A[-10, \ldots, 10][-25, \ldots, 25][-9 \ldots 29]$. $B = 0$, size $= 10$, RMO. loca^n of $A[8][20][15]$.

c_{51} c_{39}

$[8][20][15]$

$8-(-10) = 18$ $\quad 20-(-25) = 45$ $\quad 15-(-9) = 24$

$0 + [18 \cdot (51 \times 39) + 45 \cdot (39) + 24] \cdot 10$

$= 375810$

CMO: $0 + [18 \cdot (51 \cdot 39) + 24(51) + 45] \cdot 10 = 370710$

$A[l_1 \ldots u_1][l_2 \ldots u_2][l_3 \ldots u_3]$

RMO :

$A[i][j][k]$ is at $B + [(i - l_1) \cdot r \cdot c + (j - l_2) \cdot c + (k - l_3)] \cdot \text{size}$

CMO :

$B + [(i - l_1) \cdot r \cdot c + (k - l_3) \cdot r + (j - l_2)] \cdot \text{size}$

To find the maximum element in an array with the minimum # of comparisons

min # of comparisons : $(n-1)$

```
def max_element(list):
    max = list[0]
    for i in list:
        if i > max:
            max = i
    return max
```

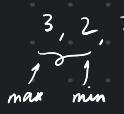
Tournament method :



This takes n-1 comparisons as well.

To find the maximum and minimum elements in an array with the minimum # of comparisons

Using linear search :



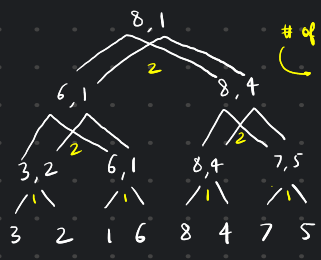
Worst # of comparisons = $2(n-2)+1 = 2n-3$



$3\left(\frac{6}{2}\right) - 2 = 7$

Tournament method :
(works best if # of elements = 2^k)

If n is a power of 2



of comparisons = 10

$$T(n) = \begin{cases} 2T(n/2) + 2 & \text{if } n > 2 \\ 1 & \text{if } n = 2 \\ 0 & \text{if } n = 1 \end{cases}$$

$$T(n) = \frac{3n}{2} - 2$$

$$= \frac{3(8)}{2} - 2 = 10$$

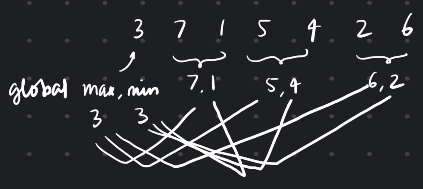
Only guarantees optimal time for powers of 2.

If n is not a power of 2 :

$$T(n) = \begin{cases} T\left(\lfloor \frac{n}{2} \rfloor\right) + T\left(\lceil \frac{n}{2} \rceil\right) + 2 \\ 1 \\ 0 \end{cases}$$

Comparison in pairs :

Odd # of elements :



For n-1 elements, we have $\frac{n-1}{2}$ pairs.

Compare both each pair. 1
Compare with global min 1
and global max 1

For odd n, $\frac{3(n-1)}{2}$ comparisons.

$\left\lceil \frac{3n}{2} - 2 \right\rceil$
for either

Even # of elements :



of pairs = $\frac{n-2}{2}$

of comparisons $= 1 + \frac{(n-2)}{2} \cdot 3$

$= \frac{3n}{2} - 2$

For even n, $\frac{3n}{2} - 2$ comparisons

Sorting

> In-place/ internal sorting:
    It uses constant extra space. It sorts the array by only modifying the orders of the elements within the list.
> Out-of-place sorting:
    It uses extra space other than the array itself.
> Stable sorting:
    When 2 same data appear in some order, then the same data appear in the same order in the sorted array as well.
    Stable sorting algorithms maintain the relative order of records with equal keys.
> Unstable sorting:
    Relative order of same elements is not preserved.

Selection sort

It repeatedly selects the smallest (or largest) element from the unsorted portion of the list and moves it to the sorted portion of the list.

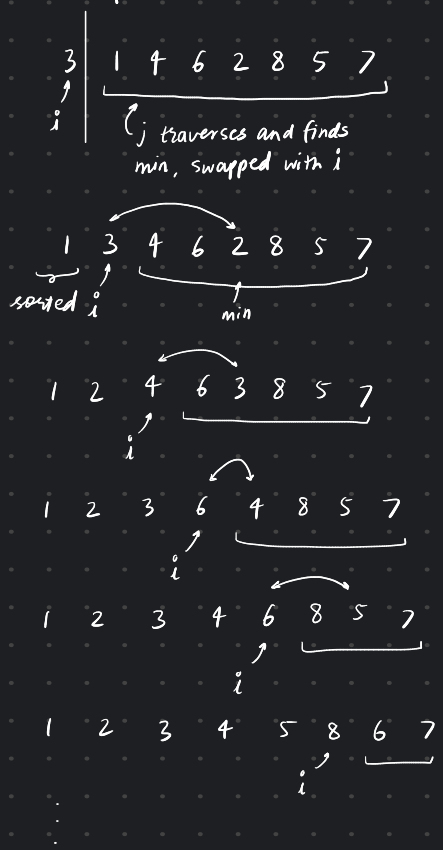
```
def selection_sort(arr, n):
    for i in range(n):
        min = arr[i]
        loc = i
        for j in range(i+1, n):
            if arr[j] < min:
                min = arr[j]
                loc = j
        arr[i], arr[loc] = arr[loc], arr[i]
    return arr
```

Time complexity:
    Best case: $O(n^2)$
    Worst case: $O(n^2)$

Extra space: $O(1)$

In-place: yes
Stable: no

i is the point till which array is sorted.
we find the min element of subarray ahead of i.
we swap it with i.



Bubble sort

It works by repeatedly swapping adjacent elements if they are in the wrong order.

```python
def bubble_sort(arr, n):
    for k in range(n):
        swapped = False
        for j in range(n-k-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]
                swapped = True
        if not swapped:
            break
    return arr
```

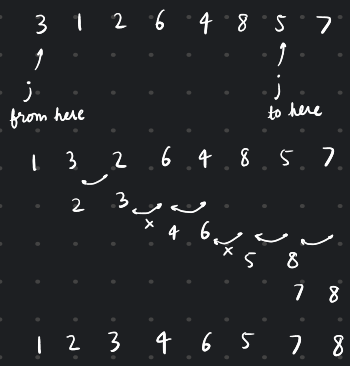
Time complexity: (only if swapped flag is used)
- Best case: $O(n)$
- Worst/average case: $O(n^2)$

Extra space: $O(1)$

In-place: yes

Stable: yes (we are not swapping same elements.)

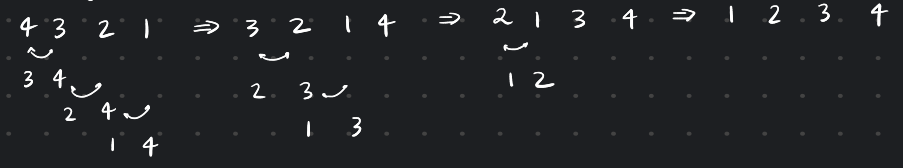
1st pass:



In each pass, the largest element will bubble up from 0 to n-k-1

Worst case:

reversed array:



Insertion sort

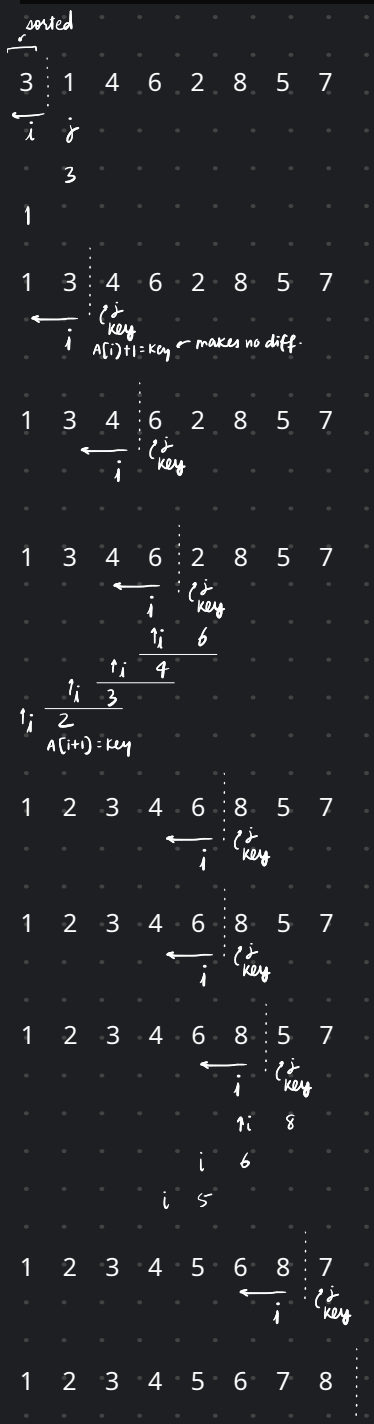
It divides the array into a sorted part and an unsorted part. It iterates through all the elements and insert each element into the sorted array at the position where it should belong.
It works iteratively inserting each element of an unsorted list into its correct position in the sorted portion of the list.

```python
def insertion_sort(arr, n):
    for j in range(1, n):
        key = arr[j]
        i = j-1
        while i >= 0 and arr[i] > key:
            arr[i+1] = arr[i]
            i -= 1
        arr[i+1] = key
    return arr
```

Time complexity:
- Best case: $O(n)$
- Worst/average case: $O(n^2)$

Extra space: $O(1)$

In-place: yes
Stable: yes



INSERTION-SORT(A)

```
1  for j = 2 to A.length
2      key = A[j]
3      // Insert A[j] into the sorted sequence A[1 .. j − 1].
4      i = j − 1
5      while i > 0 and A[i] > key
6          A[i + 1] = A[i]
7          i = i − 1
8      A[i + 1] = key
```

Bubble, selection, and insertion sort all keep a part of the array sorted.
You could think of these with complexity as

$$T(n) = T(n-1) + O(n)$$

which turns out to be $O(n^2)$.

These are called "subtract and conquer" problems.